CIFAR-10 Classification with a Compact Residual Network

Deep Learning Project 1 - Fall 2026

Author: Zihan Xu | Kaggle team: Zihan Xu

Public code and reproducible release:
https://github.com/Zihan-Xu/DL-project1

1. Overview and findings

The objective is to train a modified residual CNN from scratch using only course-provided CIFAR-10 data, with at most five million trainable parameters. This version packages the highest mean-validation configuration from a completed and independently accepted 86-run, 22-configuration combination study. It combines a widened CIFAR ResNet-20, anti-aliased downsampling, Cutout, stronger weight decay and 240 training epochs.

Measure	Result and interpretation
Trainable parameters	2,436,346, including all trainable layers; below 5,000,000
Raw validation mean	95.3867% across training seeds 42, 43, 44
Flip validation mean	95.7667% on the same 5,000 images and checkpoints
Recommended checkpoint	Fixed seed42: 95.78% (4,789/5,000) with flip averaging
Reported Kaggle result	85.00% for seed42/raw; user-reported result, supplied CSV matched locally
New/private test result	Flip score and final private-test accuracy are not available

The selected recipe is the best observed mean in this accepted study, not a statistically established global optimum. Additional experiments were still incomplete or unaccepted when this release was prepared; they are outside the selection scope. A local validation improvement does not establish a Kaggle improvement.

2. Architecture and design rationale

The network uses a 3x3 stem and three stages with 48, 96 and 192 channels. Each stage contains three two-convolution residual blocks; shortcuts are identity mappings or learned projections at stage transitions. Global average pooling feeds a ten-class linear head. Fixed BlurPool filtering precedes downsampling. The total parameter count is 2,436,346; inference costs approximately 436.1 million multiply-accumulates per image in the project counter.

Residual connections support optimization [1]. BlurPool reduces aliasing during spatial downsampling [2], at higher compute cost than the corresponding stride-only model. Widening provides capacity within the course limit. This release uses neither attention nor dropout, stochastic depth, pretrained initialization, external data or a teacher.

3. Data, augmentation and training

The provided 50,000 labeled images are split by class using seed42 into 40,000 training, 5,000 validation and 5,000 historical local-test images. Each class contributes 4,000/500/500 images. Pixel hashes were checked for exact duplicates and cross-split conflicts; no exclusions were needed. Only the training split determines RGB mean and standard deviation. The 2,000 competition images have no supplied labels and are never used for training or checkpoint selection.

Training divides uint8 pixels by 255, normalizes channels, applies random horizontal flip with probability 0.5 and Cutout with side length 12 and probability 0.5 [3]. Cutout is clipped at image boundaries and filled with normalized zero. This recipe uses no random crop, rotation, Mixup or CutMix. The registered sampler shuffles training identities with all-class batch coverage checks. Each epoch presents all 40,000 training images once.

Setting	Value
Optimizer / objective	SGD; cross-entropy; momentum 0.9; weight decay 0.0005
Schedule / budget	Initial LR 0.1; cosine decay; no warmup; batch128; 240 epochs
Numeric settings	BF16 training; FP32 validation; TF32 disabled; deterministic algorithms
Checkpoint rule	Highest raw validation correct count; ties retain the earlier epoch
Repetitions / runtime	Seeds 42,43,44; about 1,524.6 seconds training+validation per run

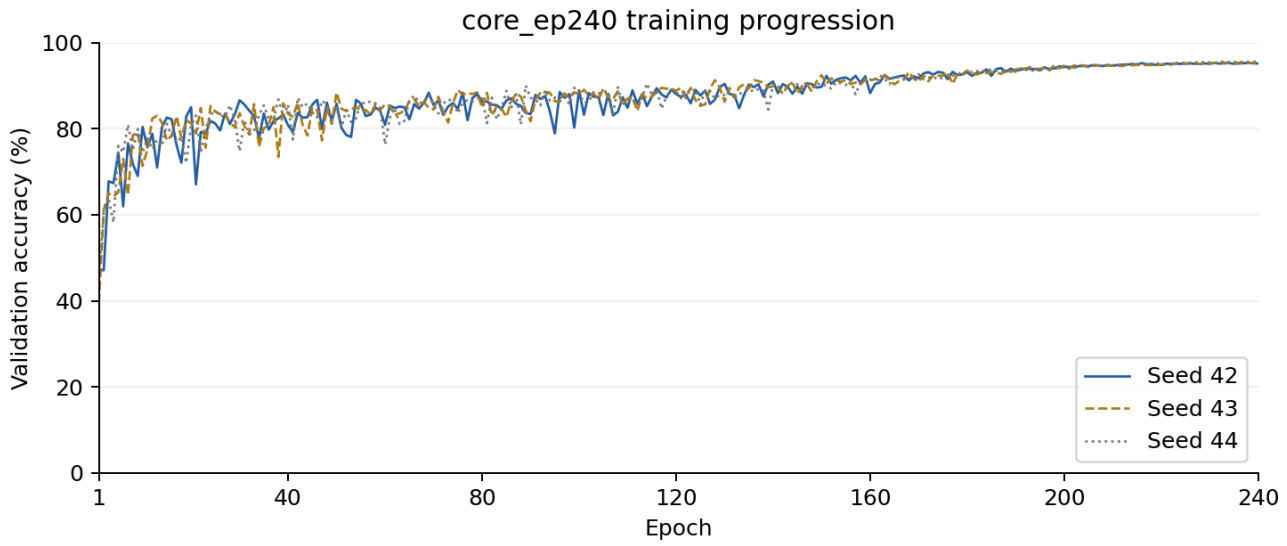


Figure 1. Raw validation progression for all three training runs on the same fixed validation split. Training and checkpoint selection were completed before packaging. Seed42 selects epoch238; seed43/44 epoch choices are in their run records.

The supplied training wrapper calls the original training implementation, preserving its augmentation, optimizer and random streams. Saved source hashes and configuration hashes identify the implementation used. Exact floating-point behavior across machines or library builds is not assumed.

4. Controlled comparisons and inference

Configuration	Seeds	Mean val. %	SD (pp)
Shared 3x baseline	5	90.3400	0.4130
BlurPool	5	92.6560	0.4046
Cutout12	5	91.7200	0.3376
WD 0.0005	5	92.2280	0.3703
Blur + Cutout + WD	5	94.6480	0.2305
Core + dropout 0.1	3	95.2000	0.2307
Core + rotation +/-10	3	95.1667	0.2663
Core, 240 epochs	3	95.3867	0.1701

All rows belong to the same completed combination study; SD describes variation across training seeds, not independent validation datasets. Core binding versus its shared baseline improved by 4.308 percentage points (five paired seeds; 95% t interval [3.8068, 4.8092]; Holm-adjusted p=0.0006033). The 240-epoch extension led the observed mean but its incremental comparison did not pass the study correction. Single-factor gains must not be added together.

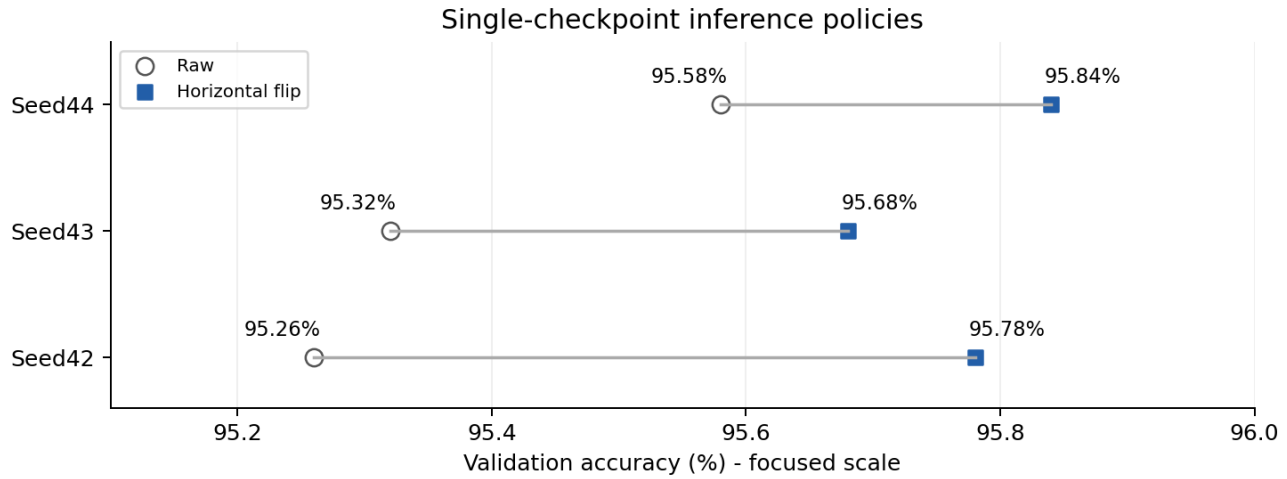


Figure 2. Each connected pair is one checkpoint evaluated with raw inference or equal averaging of original/mirrored softmax probabilities. This focused scale shows a mean gain of 0.38 percentage points; the paired-seed 95% t interval is [0.0542, 0.7058] points. It reflects three training seeds on one split and is exploratory.

The release retains seed42 as the fixed deployment choice, even though another individual seed has a higher validation result. Flip averaging uses one trained model with two forward passes. Four submission options are provided: seed42/flip (recommended), seed42/raw (reference), seed43/flip and seed44/flip (alternatives). All have exactly 2,000 predictions with the official ID order and integer classes 0-9.

5. Reproducibility and verification

The public repository contains original model and training modules, preparation and prediction entry points, pinned environment dependencies, three exported checkpoints, exact seed configurations, 240-epoch histories, validation probabilities, comparison data, prior acceptance evidence and an executed review notebook. SHA-256 manifests identify distributed files. Original course images, local caches, virtual environments and unrelated account records are excluded.

Predict: obtain authorized course data, install requirements, then run `predict.py` with `--data-dir`, the selected `--checkpoint`, `--policy flip`, and a new `--output`. The `--compare` option checks labels against the released CSV. The root README contains complete commands for each operation.

Train: run `prepare_data.py`, then `train.py --config` with the released seed config and a fresh output directory. Original training requires CUDA; prediction supports CPU. The independent report notebook checks parameters, saved probabilities, submission hashes and charts without training or hidden labels.

Packaging reloaded every included model on CPU, recomputed validation predictions and reproduced the earlier raw and flip correct counts. Every original competition PNG was read for inference. The reference seed42/raw CSV matched the existing accepted CSV byte-for-byte. New CSVs passed header, row count, uniqueness, order, class-range and digest checks. Packaging did not repeat 240-epoch training and did not submit a file to Kaggle or Brightspace.

6. Limitations and lessons

Repeated experimentation on the same validation images introduces adaptive selection risk. Repeated seeds measure training randomness, not independent-data generalization. The local validation score of roughly 95-96% contrasts with the reported raw Kaggle score of 85%; the hidden-test error cause is unproven. Neither TTA nor later experiments can be claimed to improve the leaderboard before official scoring. The private-test score is unavailable.

The strongest lesson from controlled comparisons is to verify combinations directly: their gains are not additive. Anti-aliasing and regularization are useful within this budget, but longer training and extra inference cost must be weighed against uncertain incremental gains. A smaller model may be attractive when deployment cost matters, although maximizing this release's validated accuracy remained the objective. Future accepted releases should be compared using compatible metrics and preserved as separate snapshots.

References

[1] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. CVPR, 2016. arxiv.org/abs/1512.03385

[2] R. Zhang. Making Convolutional Networks Shift-Invariant Again. ICML, 2019. arxiv.org/abs/1904.11486

[3] T. DeVries, G. W. Taylor. Improved Regularization of Convolutional Neural Networks with Cutout. 2017. arxiv.org/abs/1708.04552

[4] Course-provided Deep Learning Project 1, Fall 2026; CIFAR-10 ResNet Architecture Challenge, NYU Tandon Deep Learning F26. Course rules and deliverables were inspected from the supplied PDF and recorded authenticated competition review.